FULL PROGRAM MANIFEST

Computer Science and Computer Engineering for Games

Undergraduate foundations through master's-level specialization and thesis.

Edition: September 28, 2026 | Manifest of the existing course package

10 volumes · 50 modules · 200 lessons

200 practical lesson assignments · 600 lesson quiz questions and answers · 15 diagnostic questions · 50 module project briefs · 10 volume checkpoints · 55 references · 8 templates · 54 original archive files

What this document does

Preserves every existing module and lesson identifier; lists learning outcomes, exact module prerequisites and project briefs; collects the additional study requirements and all ten checkpoints; and inventories the reference library and every original package file.

Included material versus work to be completed

Indexes the existing package. Module projects and Gates A-J are learner requirements, not completed implementations. The ten gates overlap module project briefs and are not 10 additional independent projects. External readings and extra problem sets are assigned, not bundled textbooks or fully authored exercise banks. This manifest does not change the original curriculum or archive.

The teaching primers, worked examples, assignment prompts and quiz answers exist in the course files. Student implementations, completed capstones, independent research and the thesis remain work for the learner. Additional source exercises are assigned, not a separately authored question bank.

This is a self-study curriculum, not an accredited degree.

Navigation

Use the PDF bookmarks for the program map, each volume, additional requirements, checkpoints, templates, reference directory and file inventory. The companion JSON adds every exact practical assignment and assigned reading, plus file checksums.

Program map

Volume / checkpoint	Modules	Focus	Planning hours
01 / A	01-05	Foundations and Your First Game	220-340
02 / B	06-10	Mathematics for Games and Computing	500-800
03 / C	11-15	C++, Algorithms and Language Implementation	400-650
04 / D	16-20	Computer Systems and Software Engineering	380-600
05 / E	21-25	Graphics and Game-Engine Construction	420-650
06 / F	26-30	Game Systems and Production	420-650
07 / G	31-35	Computer Engineering and Digital Design	450-720
08 / H	36-40	Gaming Platforms and Hardware/Software Co-design	350-560
09 / I	41-45	Graduate Game-Engine and Graphics Engineering	550-850
10 / J	46-50	Graduate Gaming Hardware and Research Thesis	810-1,380

Total: 4,500-7,200 planning hours, including the labs, module work, checkpoints and one thesis. Do not add project or thesis estimates again. These are author estimates, not credits or a guaranteed schedule.

How identifiers and assessment counts work

01.1 means Module 01, Lesson 1. Each module has four lessons. Each lesson has a practical assignment and three quiz questions. Each volume therefore has 20 lessons and 60 quiz questions. Gates A-J overlap the culminating module projects; they are not ten extra independent capstones.

Default sequence

Proceed from 01.1 through 50.4, satisfying each module's prerequisites. The graduate phase requires undergraduate evidence. Use the diagnostic and prior-learning review rather than skipping material solely because the terminology looks familiar.

Volume 01 Foundations and Your First Game

Beginner / early undergraduate

Learn to reason about programs, run code, debug, test, use Git and complete a tiny game.

Modules 01-05 | 20 lessons | 60 quiz questions | Checkpoint A

Module 01: Starting as an engineer

Prerequisites: None. Basic reading, arithmetic and the ability to save a file.

ID	Lesson	Learning outcome
01.1	What a program actually does	Explain state, instructions, input and output using a tiny game.
01.2	Files, terminals and reproducible execution	Run a saved Python program and distinguish a shell from an interpreter.
01.3	Trace before running	Predict a program by following assignments in order.
01.4	Units, budgets and engineering notebooks	Use units and simple arithmetic to check game calculations.

Module project: Create a learning repository with a working motion program, an environment record, a prediction table and a one-page explanation of the input-to-display path.

References: R03, R04, R15 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 02: Values, decisions and repetition

Prerequisites: Module 01.

ID	Lesson	Learning outcome
02.1	Types and expressions	Choose integers, real-valued numbers, strings and booleans appropriately.
02.2	Conditions and boundary cases	Write decisions with complete and non-overlapping rules.
02.3	Loops and invariants	Use repetition that terminates and preserves a stated property.
02.4	Functions and contracts	Separate a calculation from interaction and state its valid inputs.

Module project: Build a terminal-based resource-gathering simulation with bounded health, inventory and a fixed number of turns. Include invalid-input handling and a complete trace for a five-turn run.

References: R03, R04 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 03: Organizing data and small programs

Prerequisites: Module 02.

ID	Lesson	Learning outcome
03.1	Sequences, dictionaries and identity	Choose collections according to the operations the program needs.
03.2	Decomposition, modules and interfaces	Divide a small game into parts that can be tested independently.
03.3	Files, serialization and validation	Save structured data without assuming every file is trustworthy.
03.4	Classes, composition and state machines	Represent behavior without creating an unnecessary inheritance hierarchy.

Module project: Build a data-driven turn simulator with functions, a list of entities, a validated configuration file and save/load round-trip tests.

References: R03, R04 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 04: Debugging, testing and collaboration

Prerequisites: Modules 01-03.

ID	Lesson	Learning outcome
04.1	Debugging as an experiment	Find a defect by narrowing evidence rather than making random edits.
04.2	Unit, integration and property tests	Choose tests that challenge the contract instead of repeating the implementation.
04.3	Git, branches and recoverable changes	Use small commits to preserve reasoning and collaborate safely.
04.4	Review, documentation and responsible AI help	Explain a change and verify assistance without surrendering understanding.

Module project: Publish a local tagged release of the terminal simulation with tests, a reproducible bug report, a reviewed change and an evidence-based development journal. A remote public upload is not required.

References: R03, R05, R11 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 05: A complete first game

Prerequisites: Modules 01-04.

ID	Lesson	Learning outcome
05.1	The update loop and command model	Separate simulation ticks from input collection and presentation.
05.2	Collision rules and complete behavior	Specify edge cases before implementing a playable rule.
05.3	Deterministic replay and simple measurement	Reproduce a run and measure the work actually performed.
05.4	Finishing, usability and the first portfolio artifact	Deliver a tiny game another person can run and understand.

Module project: Complete Gate A: a deterministic terminal grid game with movement, obstacles, win/loss states, replay, tests and a clean-start README. Use the detailed project brief in the program handbook.

References: R20, R22, R27, R29 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 02 Mathematics for Games and Computing

Undergraduate mathematical core

Build the mathematical language for algorithms, graphics, physics, signals and experiments.

Modules 06-10 | 20 lessons | 60 quiz questions | Checkpoint B

Module 06: Discrete mathematical reasoning

Prerequisites: Volume 01; complete the algebra diagnostic in START_HERE before proceeding.

ID	Lesson	Learning outcome
06.1	Algebra, functions, logarithms and trigonometry	Translate a game relationship into an equation with a defined domain.
06.2	Logic, sets and quantified claims	Express requirements precisely enough to test or prove.
06.3	Proofs, induction and invariants	Justify a program property instead of testing only examples.
06.4	Counting, graphs and recurrences	Model combinations and recursive work without brute-force guessing.

Module project: Write a mathematical notebook containing a truth-table specification, an induction proof, a graph model and a justified operation-count estimate. Include counterexamples to three false claims.

References: R06, R08 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 07: Linear algebra and game geometry

Prerequisites: Module 06.1; basic Python functions.

ID	Lesson	Learning outcome
07.1	Vectors, dot products and projections	Use vectors for direction, distance and component measurements.
07.2	Matrices, bases and linear systems	Understand a matrix as a transformation rather than a table to memorize.
07.3	Homogeneous transforms, cameras and quaternions	Compose a scene using explicit coordinate conventions.
07.4	Eigenvalues, least squares and singular values	Recognize stability and approximation problems in game data.

Module project: Build a tested vector/matrix library for a two-dimensional scene, then demonstrate a three-dimensional camera transformation with written coordinate conventions.

References: R07, R19, R23 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 08: Calculus, mechanics and numerical simulation

Prerequisites: Modules 06.1 and 07.1; work through the linked calculus problem sets.

ID	Lesson	Learning outcome
08.1	Limits, derivatives and rates	Connect derivatives to velocity, acceleration and local approximation.
08.2	Integrals, accumulation and continuous probability	Interpret integration as accumulated contribution.
08.3	Newtonian mechanics and differential equations	Model motion from forces with clear simplifying assumptions.
08.4	Time integration, stability and error	Choose and test a numerical update rather than assuming smaller code is correct.

Module project: Build a projectile and spring simulator with analytic comparison cases, timestep refinement, unit checks and a report separating model error from numerical error.

References: R08, R23, R28 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 09: Probability, statistics and evidence

Prerequisites: Modules 06 and 08.2.

ID	Lesson	Learning outcome
09.1	Probability and conditional reasoning	Model uncertainty without confusing independent and conditional events.
09.2	Random variables, distributions and sampling	Distinguish an expected value from a guaranteed outcome.
09.3	Inference, uncertainty and benchmark design	Compare implementations without hiding variance or bias.
09.4	Regression, correlation and ethical telemetry	Fit a model while recognizing confounding and data limits.

Module project: Create a reproducible benchmark experiment and a loot-distribution simulation. Report assumptions, uncertainty, representative workloads and one result that did not support the original hypothesis.

References: R09, R06 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 10: Signals, optimization and numerical foundations

Prerequisites: Modules 07-09.

ID	Lesson	Learning outcome
10.1	Multivariable calculus and gradients	Use partial derivatives to reason about spatial fields and optimization.
10.2	Sampling, aliasing and Fourier ideas	Explain why discrete samples can misrepresent a continuous signal.
10.3	Convolution, filters and systems	Predict a filter's effect on data and latency.
10.4	Optimization, conditioning and approximation budgets	Choose an objective and distinguish convergence from correctness.

Module project: Complete Gate B: a mathematical engineering notebook plus tested transforms, numerical integration, signal filtering and an uncertainty-aware experiment.

References: R07, R10, R23 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 03 C++, Algorithms and Language Implementation

Undergraduate CS core

Write native software safely, select data structures, analyze algorithms and build a tiny language.

Modules 11-15 | 20 lessons | 60 quiz questions | Checkpoint C

Module 11: Native programming with C++

Prerequisites: Volume 01, Modules 06-07; do not skip the ownership module that follows.

ID	Lesson	Learning outcome
11.1	Compilation, linking and build systems	Explain how source code becomes a runnable native program.
11.2	Integer, floating-point and object representations	Choose native types with explicit range and precision assumptions.
11.3	Values, references, pointers and lifetimes	Track where an object lives and how long access remains valid.
11.4	Standard containers, iterators and error handling	Use library abstractions while respecting invalidation and failure contracts.

Module project: Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

References: R11, R12, R15 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 12: Ownership, abstractions and C++ design

Prerequisites: Module 11.

ID	Lesson	Learning outcome
12.1	RAII and resource ownership	Make cleanup follow object lifetime on success and failure paths.
12.2	Copy, move and exception safety	Define what transferring a resource means without duplicating ownership.
12.3	Interfaces, composition and polymorphism	Choose abstractions according to substitutability and measured cost.
12.4	Templates, concepts and data-oriented choices	Use generic code without hiding costs or exploding complexity.

Module project: Build a small resource manager with explicit ownership, safe move behavior, bounded interfaces, fault-injection tests and a written comparison of composition and inheritance.

References: R11, R12 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 13: Data structures and their costs

Prerequisites: Modules 06 and 11-12.

ID	Lesson	Learning outcome
13.1	Complexity, arrays and linked structures	Predict asymptotic work while recognizing locality and constants.
13.2	Stacks, queues and ring buffers	Match ordering semantics to the intended workflow.
13.3	Hash tables and collision policies	Explain expected lookup behavior and its assumptions.
13.4	Trees, heaps and priority queues	Preserve structural invariants and choose the right query behavior.

Module project: Implement educational versions of a dynamic array, queue, hash table and heap, each with invariants and tests. Compare them to standard containers without replacing standard code in production merely for practice.

References: R13, R06 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 14: Algorithms for maps, simulation and tools

Prerequisites: Modules 06, 09 and 13.

ID	Lesson	Learning outcome
14.1	Searching, sorting and algorithm selection	Choose an algorithm from its preconditions and the data distribution.
14.2	Graph traversal and dependency ordering	Use graph algorithms with explicit edge semantics.
14.3	Dijkstra, A* and admissible heuristics	Build a correct pathfinder before optimizing its queue.
14.4	Dynamic programming, complexity limits and approximation	Recognize repeated subproblems and justify a practical solution.

Module project: Build a pathfinding visualizer and asset-dependency planner. Provide correctness arguments, unreachable cases, complexity analysis and measured comparisons on multiple map families.

References: R13, R06 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 15: Programming languages, compilers and computability

Prerequisites: Modules 11-14 and discrete proofs.

ID	Lesson	Learning outcome
15.1	Language paradigms and type systems	Separate language semantics from syntax and implementation habits.
15.2	Lexing, parsing and syntax trees	Convert source text into a structured representation with diagnostics.
15.3	Bytecode, virtual machines and bounded execution	Implement a small execution model with explicit limits.
15.4	Automata, computability and compiler optimization	Know what analysis can establish and when transformation is legal.

Module project: Complete Gate C: a tested C++ simulation plus a tiny expression language with lexer, parser, evaluator, bytecode execution and resource limits.

References: R14, R06, R11 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 04 Computer Systems and Software Engineering

Intermediate undergraduate

Connect programs to processors, operating systems, concurrency, networks, storage and professional practice.

Modules 16-20 | 20 lessons | 60 quiz questions | Checkpoint D

Module 16: Processor architecture and assembly

Prerequisites: Modules 11-15.

ID	Lesson	Learning outcome
16.1	Instruction sets, registers and calling conventions	Read a small machine-level routine without confusing ISA and implementation.
16.2	Pipelines, dependencies and branch prediction	Explain why instruction count alone does not predict latency.
16.3	Caches, locality and memory hierarchy	Relate access patterns to cache lines and working sets.
16.4	Virtual memory, SIMD and architectural limits	Recognize address translation and vector execution as separate mechanisms.

Module project: Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

References: R15, R35 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 17: Operating systems and resource management

Prerequisites: Module 16.

ID	Lesson	Learning outcome
17.1	Processes, threads, system calls and protection	Explain which resources are isolated and which are shared.
17.2	Allocation, paging and memory-mapped data	Distinguish reserved address space, committed storage and live objects.
17.3	Scheduling, responsiveness and real-time constraints	Distinguish average completion from meeting a deadline.
17.4	Filesystems, persistence and asynchronous I/O	Design data operations that survive partial failure.

Module project: Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

References: R16, R15 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 18: Concurrency and parallel programming

Prerequisites: Modules 12 and 16-17.

ID	Lesson	Learning outcome
18.1	Races, mutexes and happens-before	Protect shared invariants using a clear synchronization model.
18.2	Condition variables, deadlocks and cancellation	Make waiting and shutdown correct under adverse timing.
18.3	Atomics, memory ordering and false sharing	Use low-level synchronization only with a complete correctness argument.
18.4	Task graphs, work stealing and scaling limits	Parallelize independent work and measure end-to-end benefit.

Module project: Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

References: R11, R16, R37 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 19: Networks, storage and data services

Prerequisites: Modules 15 and 17-18.

ID	Lesson	Learning outcome
19.1	Network layers, latency and transport semantics	Choose transport behavior from application requirements.
19.2	Protocol design, serialization and timeouts	Treat every received byte as untrusted structured input.
19.3	Relational modeling, SQL and indexes	Represent game data with constraints that survive application mistakes.
19.4	Transactions, consistency and distributed failure	Preserve invariants when several operations must succeed together.

Module project: Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

References: R17, R18, R16 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 20: Engineering practice, security and maintainability

Prerequisites: Volumes 01-04.

ID	Lesson	Learning outcome
20.1	Requirements, architecture and design records	Make tradeoffs explicit and trace implementation back to needs.
20.2	Defensive programming and threat modeling	Protect assets and trust boundaries before selecting security mechanisms.
20.3	Build automation, dependencies and release discipline	Produce a verifiable artifact from a known source revision.
20.4	Ethics, accessibility, teamwork and sustainable delivery	Evaluate engineering decisions beyond code correctness.

Module project: Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

References: R01, R02, R05, R11, R29 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 05 Graphics and Game-Engine Construction

Upper undergraduate

Build a renderer and a small engine instead of only learning an existing editor.

Modules 21-25 | 20 lessons | 60 quiz questions | Checkpoint E

Module 21: From geometry to a software renderer

Prerequisites: Modules 07-08, 11-16; complete the mathematics checkpoint.

ID	Lesson	Learning outcome
21.1	Images, color and coordinate conventions	Define the numerical and geometric meaning of an image buffer.
21.2	Projection and clipping before perspective division	Transform geometry without confusing visibility with division by depth.
21.3	Triangle coverage, depth and perspective-correct interpolation	Produce crack-resistant filled triangles with correct varying attributes.
21.4	Rays, intersections and acceleration structures	Build a second visibility method and compare its costs with rasterization.

Module project: Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

References: R19, R23 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 22: Explicit GPU programming and graphics APIs

Prerequisites: Module 21 plus Modules 18 and 20. Choose one compatible backend, not all at once.

ID	Lesson	Learning outcome
22.1	GPU work submission, resources and pipeline state	Separate CPU recording from asynchronous GPU execution.
22.2	Buffers, shaders, layouts and resource binding	Match data representations across CPU and GPU boundaries.
22.3	Synchronization, hazards and frames in flight	Express dependencies without serializing the entire renderer.
22.4	Presentation, resizing and device capability policy	Treat the window system as a changing environment rather than a fixed image.

Module project: Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

References: R21, R22, R24, R25, R37 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 23: Lighting, materials and real-time image quality

Prerequisites: Modules 07, 09-10, 21-22.

ID	Lesson	Learning outcome
23.1	Radiometry, reflectance and physically based materials	Explain how lighting models approximate light transport.
23.2	Textures, mipmaps, filtering and tangent-space detail	Control aliasing while preserving the meaning of texture data.
23.3	Shadows, light selection and rendering-path tradeoffs	Choose a lighting architecture from scene and bandwidth requirements.

ID	Lesson	Learning outcome
23.4	HDR, tone mapping, temporal reconstruction and validation	Separate display mapping from reconstruction and simulation correctness.

Module project: Create a material-and-lighting demonstration with a documented color pipeline. Compare image quality and frame cost at identical resolution and scene content; include reference images, not only attractive screenshots.

References: R19, R23, R21 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 24: Engine architecture, simulation and resource ownership

Prerequisites: Modules 12, 17-18, 21-23.

ID	Lesson	Learning outcome
24.1	Fixed updates, interpolation and overload policy	Separate simulation time from rendering time and real elapsed time.
24.2	Entities, components, handles and data-oriented storage	Design identity separately from memory location.
24.3	Allocators, task graphs and frame-resource lifetime	Make temporary storage and parallel dependencies observable.
24.4	Subsystem boundaries, save formats and reloadability	Choose interfaces that make change and failure manageable.

Module project: Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

References: R20, R26, R27, R11 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 25: Asset pipelines, editors and an integrated engine

Prerequisites: Module 24 and the software-engineering practices of Module 20.

ID	Lesson	Learning outcome
25.1	Importing, cooking and content-addressed asset builds	Treat assets as compiled inputs with dependency tracking.
25.2	Streaming, residency and memory budgets	Load content without allowing asynchronous work to violate ownership.
25.3	Editors, inspection and undoable commands	Make engine state understandable and safely editable.
25.4	Engine integration and reproducible demonstration	Deliver a small complete system whose limitations are measurable.

Module project: Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

References: R20, R26, R22 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 06 Game Systems and Production

Upper undergraduate / engineering capstone

Add simulation, animation, audio, AI, multiplayer, accessibility and release discipline.

Modules 26-30 | 20 lessons | 60 quiz questions | Checkpoint F

Module 26: Collision detection and physical simulation

Prerequisites: Modules 07-10, 14 and 24.

ID	Lesson	Learning outcome
26.1	Broad phase, narrow phase and geometric robustness	Separate possible collisions from confirmed contacts.
26.2	Rigid bodies, angular motion and collision impulses	Use conservation laws with explicit approximations.
26.3	Constraints, iterative solvers and continuous collision detection	Explain where discrete simulation fails and how solvers control error.
26.4	Physics testing, determinism and engine integration	Validate simulation with invariants and adversarial scenes.

Module project: Build a deterministic test harness for a small physics scene, compare your simple implementation with a maintained physics library, and record precisely which physical and numerical assumptions differ.

References: R28, R19, R23 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 27: Animation, audio and interactive timing

Prerequisites: Modules 07, 10, 18 and 24.

ID	Lesson	Learning outcome
27.1	Skeletons, transforms and skinning	Compute hierarchical animation without mixing coordinate spaces.
27.2	Animation blending, state machines and inverse kinematics	Coordinate authored motion with runtime goals.
27.3	Digital audio, mixing and real-time callback discipline	Generate sound without missing the output deadline.
27.4	Spatial audio, animation events and multimodal accessibility	Synchronize feedback while preserving user control.

Module project: Integrate an animated character or articulated object with event-driven audio. Demonstrate smooth interpolation, bounded audio work, adjustable sound categories and correct behavior when animation rate differs from simulation rate.

References: R19, R20, R22, R26 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 28: Game AI, decision systems and machine learning

Prerequisites: Modules 09, 14, 24 and 26.

ID	Lesson	Learning outcome
28.1	Navigation graphs, steering and dynamic obstacles	Separate finding a route from executing movement.
28.2	Behavior trees, utility systems and explainable decisions	Represent behavior so designers can understand its failures.
28.3	Adversarial search, planning and simulation budgets	Trade decision quality against bounded computation.

ID	Lesson	Learning outcome
28.4	Learning systems, neural inference and responsible evaluation	Understand learning fundamentals without replacing every subsystem with a model.

Module project: Build an inspectable agent that navigates, chooses actions and recovers from blocked plans. Compare a rules-based baseline with one small learned or search-based component; evaluate behavior and runtime cost, not only win rate.

References: R20, R46, R47 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 29: Multiplayer simulation and online services

Prerequisites: Modules 18-20 and 24; Module 26 for physical interaction.

ID	Lesson	Learning outcome
29.1	Authority, prediction and reconciliation	Provide responsive local control without trusting arbitrary client outcomes.
29.2	Snapshots, interpolation, relevance and lag compensation	Manage bandwidth and latency while making fairness assumptions explicit.
29.3	Lockstep, rollback and deterministic replay	Choose synchronization around the game model rather than genre slogans.
29.4	Match services, identity, persistence and failure recovery	Design a small online service without treating it as part of the render loop.

Module project: Create a local two-client multiplayer demonstration with a network simulator for delay, jitter, loss, duplication and reordering. No public server or paid cloud account is required. Document authority and recovery rules.

References: R17, R20, R26 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 30: Production quality and the undergraduate engine capstone

Prerequisites: Modules 21-29 plus Gate D software-engineering evidence.

ID	Lesson	Learning outcome
30.1	Profiling, frame budgets and performance regression tests	Optimize measured bottlenecks without sacrificing correctness.
30.2	Compatibility, accessibility, localization and human testing	Test whether people can actually use the complete product.
30.3	Packaging, patching, modding and supply-chain boundaries	Distribute a game without creating an uncontrolled execution path.
30.4	Capstone review, release and technical postmortem	Demonstrate ownership of a complete engineering result.

Module project: Gate F: release a small original game using your engine, with one complete gameplay loop, settings, save/load, diagnostics, accessibility checks, a tested distribution and a technical postmortem. Scope is judged by quality of evidence, not visual size.

References: R20, R26, R29, R45 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 07 Computer Engineering and Digital Design

Undergraduate CE core

Move from circuits and gates to RTL, a processor and an FPGA-based game subsystem.

Modules 31-35 | 20 lessons | 60 quiz questions | Checkpoint G

Module 31: Circuits, measurement and electronic foundations

Prerequisites: Modules 06-10; basic algebra and calculus competence required. Simulation is the default laboratory.

ID	Lesson	Learning outcome
31.1	Voltage, current, resistance and safe measurement	Analyze a simple circuit using units and conservation laws.
31.2	Capacitors, inductors, transients and frequency response	Relate stored energy to timing and filtering behavior.
31.3	Diodes, transistors, CMOS and analog interfaces	Understand the physical devices beneath digital abstractions.
31.4	Sampling, ADCs, DACs and noisy controller inputs	Convert physical signals into stable digital measurements.

Module project: Create a circuit laboratory notebook using simulation first: voltage/current prediction, an RC transient, a transistor switch and a sampled input. Physical extensions require a qualified mentor and a documented low-voltage safety review.

References: R30, R40, R10 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 32: Digital logic, state and timing

Prerequisites: Modules 06, 15-16 and 31.

ID	Lesson	Learning outcome
32.1	Boolean logic and combinational building blocks	Translate a logical specification into a circuit with no stored state.
32.2	Registers, finite-state machines and synchronous timing	Model state transitions at clock boundaries.
32.3	Binary arithmetic, fixed point and overflow policies	Choose representations that preserve the needed range and precision.
32.4	Clock domains, reset and metastability	Recognize where synchronous assumptions stop applying.

Module project: Build and test an arithmetic unit, a finite-state controller and a synchronous queue model. For every block, provide a truth table or transition specification before coding its implementation.

References: R15, R31, R35 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 33: SystemVerilog and verification-driven RTL design

Prerequisites: Module 32; Python and C++ foundations complete.

ID	Lesson	Learning outcome
33.1	RTL semantics and synthesizable SystemVerilog	Write a hardware description rather than a sequential software imitation.
33.2	Testbenches, scoreboards and constrained stimulus	Check behavior against an independent specification.

ID	Lesson	Learning outcome
33.3	Assertions, coverage and formal reasoning	Distinguish observed tests from properties proven under assumptions.
33.4	Synthesis, constraints and FPGA resource mapping	Connect verified behavior to a realizable implementation.

Module project: Implement a parameterized synchronous FIFO in RTL with a Python or C++ reference model, automated tests, assertions and synthesis reports. Keep testbench behavior separate from synthesizable hardware.

References: R32, R33, R34, R50 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 34: Designing a small processor

Prerequisites: Modules 16 and 32-33. Implement a documented subset, not an unqualified full-ISA claim.

ID	Lesson	Learning outcome
34.1	Instruction sets, datapaths and single-cycle execution	Separate programmer-visible behavior from its implementation.
34.2	Pipelining, forwarding, stalls and branch recovery	Preserve architectural behavior while overlapping instructions.
34.3	Memory interfaces, caches and protection boundaries	Design memory behavior as a protocol rather than an instantaneous array.
34.4	Processor verification, toolchains and architectural evidence	Demonstrate instruction correctness with reproducible programs and traces.

Module project: Create a simple processor model and RTL implementation for a clearly specified instruction subset. Run the same short programs on an instruction-level reference and the RTL, comparing architectural state at retirement.

References: R15, R31, R35, R32 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 35: FPGA-based game subsystems and integration

Prerequisites: Modules 21, 31-34. Physical deployment is optional; simulated I/O satisfies the core project.

ID	Lesson	Learning outcome
35.1	Memory-mapped I/O, timers, interrupts and DMA	Explain how software controls asynchronous hardware.
35.2	Framebuffers, scanout and pixel pipelines	Produce timed pixel data with correct storage and scheduling.
35.3	A small accelerator and its software contract	Measure whether specialized hardware improves a complete operation.
35.4	Integrated digital-design review and optional board bring-up	Close the loop between specification, verification and implementation.

Module project: Gate G: demonstrate a verified RTL game subsystem, such as a sprite compositor, controller interface or bounded vector unit. Include a software reference, interface specification, automated verification and synthesis evidence. A physical board is an extension, not a prerequisite.

References: R15, R32, R33, R34, R35 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 08 Gaming Platforms and Hardware/Software Co-design

Upper undergraduate CE / capstone

Study boards, memory, GPU architecture, performance, input/display pipelines and integrated design.

Modules 36-40 | 20 lessons | 60 quiz questions | Checkpoint H

Module 36: Embedded platforms, drivers and board design

Prerequisites: Modules 17-18 and 31-35. Physical work remains optional and supervised.

ID	Lesson	Learning outcome
36.1	Serial interfaces and peripheral contracts	Choose a communication interface using timing and electrical constraints.
36.2	Firmware, RTOS scheduling and driver lifecycle	Bound embedded work and make device state transitions explicit.
36.3	Schematics, PCB layout, power integrity and signal integrity	Translate a circuit into a manufacturable layout without ignoring physics.
36.4	Bring-up, instrumentation and fault isolation	Turn an unresponsive device into a sequence of testable hypotheses.

Module project: Design a simulated controller or peripheral with firmware, a host interface and a schematic-level review. A physical board extension begins only after the electrical, layout and bring-up plans receive qualified review.

References: R36, R51, R53, R30 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 37: GPU microarchitecture and memory systems

Prerequisites: Modules 16, 22, 32 and 34.

ID	Lesson	Learning outcome
37.1	SIMT execution, divergence, occupancy and latency hiding	Explain why many threads do not guarantee useful parallel work.
37.2	Coalescing, local storage, caches and shared-memory banks	Match data layout to the memory transactions actually performed.
37.3	Raster, texture, matrix and ray-tracing units	Understand fixed-function acceleration as specialized tradeoffs.
37.4	CPU-GPU sharing, discrete memory and synchronization	Treat shared addressability separately from ownership and cost.

Module project: Create a GPU architecture workbook connecting shader behavior to execution groups, registers, memory transactions and fixed-function resources. Measure one accessible GPU or clearly label a simulator-only study.

References: R37, R48, R43, R21 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 38: Quantitative performance engineering

Prerequisites: Modules 09, 16, 18, 30 and 37.

ID	Lesson	Learning outcome
38.1	Amdahl, roofline and bottleneck models	Use upper bounds to reject implausible optimization claims.
38.2	Profilers, counters and trustworthy benchmark design	Distinguish observation from an explanation that still needs testing.
38.3	SIMD, cache blocking, NUMA and job scheduling	Optimize locality and parallelism together.

ID	Lesson	Learning outcome
38.4	Energy, thermal behavior and sustained performance	Evaluate useful work over time rather than only a short peak.

Module project: Produce a performance investigation with a hypothesis, controlled workload, raw measurements, uncertainty, a bottleneck model and a verified optimization. A negative result is acceptable when the evidence is rigorous.

References: R43, R37, R48, R26 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 39: Gaming-platform constraints and interactive latency

Prerequisites: Modules 22, 27, 29 and 36-38.

ID	Lesson	Learning outcome
39.1	Input-to-display latency and frame pacing	Account for the whole interaction path rather than only frame rate.
39.2	Mobile and handheld systems: memory, power and lifecycle	Design for suspension, limited budgets and sustained use.
39.3	XR spaces, stereo views, pose timing and comfort constraints	Render from runtime-provided timing and coordinate contracts.
39.4	Portability, platform services and console-development boundaries	Build transferable systems without assuming proprietary access.

Module project: Create a platform capability and latency report for the engine. Implement one non-desktop constraint in simulation or on an available device: touch input, suspend/resume, restricted memory, variable presentation or XR timing.

References: R22, R24, R25, R29, R52, R53 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 40: Hardware/software co-design capstone

Prerequisites: Gates F and G; Modules 36-39.

ID	Lesson	Learning outcome
40.1	Workload characterization and measurable requirements	Derive hardware goals from a real software operation.
40.2	Architecture alternatives, arithmetic precision and dataflow	Explore several designs before committing to one.
40.3	Interface design, virtualization and end-to-end evaluation	Include the host software in the hardware result.
40.4	Design review, evidence tiers and undergraduate CE completion	Defend a co-design decision with limitations and reproducible artifacts.

Module project: Gate H: choose one game workload and compare a software baseline with a specialized hardware or architecture alternative. Deliver a reference implementation, model or RTL, software interface, verification evidence and an end-to-end tradeoff report.

References: R43, R48, R49, R38, R33 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 09 Graduate Game-Engine and Graphics Engineering

Master's-level specialization

Derive algorithms, reproduce research, optimize parallel systems and defend measured engineering choices.

Modules 41-45 | 20 lessons | 60 quiz questions | Checkpoint I

Module 41: Graduate numerical methods and research practice

Prerequisites: Gates B, F and H; rigorous linear algebra, calculus, probability and systems competence.

ID	Lesson	Learning outcome
41.1	Reading research as claims, assumptions and evidence	Extract an implementable argument rather than only a paper summary.
41.2	Conditioning, sparse systems and iterative solvers	Choose a solver from mathematical properties and error requirements.
41.3	Constrained optimization, derivatives and optimality conditions	Derive and verify an optimization method for an engineering objective.
41.4	Experimental design, uncertainty and reproducible graduate work	Make a performance or quality claim that survives independent scrutiny.

Module project: Produce a graduate numerical notebook with derivations, reference implementations, conditioning experiments and a preregistered evaluation plan. Complete substantial assigned problem sets; the short lesson quizzes are only formative checks.

References: R42, R54, R09, R23 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 42: Advanced parallel and GPU algorithms

Prerequisites: Modules 18, 37-38 and 41.

ID	Lesson	Learning outcome
42.1	Parallel reductions, scans and work-span analysis	Prove the structure of a parallel primitive before tuning it.
42.2	Tiling, data reuse and asynchronous pipelines	Overlap movement and computation without creating races.
42.3	Irregular workloads, wavefront scheduling and load balance	Design around variable work rather than assuming identical threads.
42.4	Portable performance and integration-quality evaluation	Separate algorithmic benefit from one device-specific tuning win.

Module project: Implement a reusable GPU reduction, scan or compaction primitive and integrate it into an engine workload. Compare a simple reference, a competent library baseline when available and your implementation at equal correctness requirements.

References: R37, R43, R48 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 43: Advanced rendering and reconstruction

Prerequisites: Modules 09-10, 21-23 and 41-42.

ID	Lesson	Learning outcome
43.1	Monte Carlo light transport and multiple importance sampling	Derive estimators with correct densities and measures.

ID	Lesson	Learning outcome
43.2	Real-time ray tracing, dynamic acceleration and hybrid pipelines	Budget traversal, building and shading as one system.
43.3	Spatiotemporal reuse, reservoirs and denoising	Evaluate variance reduction together with bias, history validity and motion.
43.4	Differentiable and neural rendering under engineering constraints	Test learned or gradient-based methods against explicit baselines.

Module project: Reproduce a bounded rendering result and add one justified extension. Derive the estimator or reconstruction objective, compare equal-quality cost, include temporal failure cases and preserve source scenes and evaluation scripts.

References: R23, R44, R47 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 44: Advanced engine systems and simulation research

Prerequisites: Gates F-H; Modules 41-43.

ID	Lesson	Learning outcome
44.1	Large-world scheduling, streaming and critical-path control	Design a bounded system under changing content and workload.
44.2	Deformable simulation, variational models and solver tradeoffs	Derive a physical or constraint model before comparing visual results.
44.3	Distributed game state, consistency and overload behavior	Specify which guarantees survive network delay and partial failure.
44.4	Formal specifications, replay and reliability-oriented engine design	Use precise invariants to reduce the space of subtle failures.

Module project: Conduct an engine-system experiment with a formal model or numerical argument, a reference implementation and stress-tested integration. Choose one main depth area: world streaming, simulation, distributed state or reliability.

References: R20, R26, R54, R55, R50 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 45: Compilers, shader languages and engine specialization

Prerequisites: Modules 15-16, 18, 37 and 41-42.

ID	Lesson	Learning outcome
45.1	Intermediate representations, SSA and dataflow analysis	Represent program meaning so transformations can be justified.
45.2	Alias analysis, vectorization and legal optimization	Separate transformations that are fast from transformations that preserve meaning.
45.3	Domain-specific languages, specialization and safe JIT execution	Trade generality for predictable game-engine computation.
45.4	Compiler/hardware co-design and graduate artifact review	Connect a transformation to architecture and application evidence.

Module project: Gate I: deliver a research-quality reproduction and extension in rendering, simulation, GPU algorithms or compilation. This module also requires a small compiler optimization with semantic tests, generated-code inspection and end-to-end measurements.

References: R14, R11, R37, R48 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Volume 10 Graduate Gaming Hardware and Research Thesis

Master's-level specialization / thesis

Evaluate advanced architectures, physical design and verification, then complete and defend a research artifact.

Modules 46-50 | 20 lessons | 60 quiz questions | Checkpoint J

Module 46: Advanced computer architecture for gaming workloads

Prerequisites: Modules 34, 37-40 and 41-42.

ID	Lesson	Learning outcome
46.1	Cache coherence, memory consistency and synchronization	Separate agreement about a location from ordering across locations.
46.2	Out-of-order execution, speculation and precise state	Increase instruction overlap without changing architectural outcomes.
46.3	Architecture simulation, traces and model validation	Use simulation as evidence with a stated fidelity boundary.
46.4	Memory systems, chiplets and multi-objective design	Choose an architecture across performance, power, cost and complexity.

Module project: Perform an architecture study using a pinned simulator or a carefully bounded analytical model. Compare at least three configurations on representative and adverse game-related workloads, and validate the model against a known reference where possible.

References: R48, R38, R35, R43 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 47: VLSI, timing and physical implementation

Prerequisites: Modules 31-34, 40 and 46. Use educational libraries and licensed or open tools only.

ID	Lesson	Learning outcome
47.1	CMOS logic, transistor sizing and delay models	Connect gate-level choices to electrical load and timing.
47.2	Static timing analysis, clocks and timing exceptions	Check every required path under the correct timing contract.
47.3	Synthesis, placement, clock trees, routing and physical checks	Understand how a logical netlist becomes a physical layout.
47.4	Power, area, process corners and design-space closure	Report PPA with enough assumptions to make comparison meaningful.

Module project: Carry a verified small RTL block through an educational synthesis and physical-design flow. Report constraints, timing, area and power-estimation assumptions. The result is a learning artifact, not fabrication-ready commercial signoff.

References: R49, R39, R30, R33 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 48: Verification, reliability and hardware trust boundaries

Prerequisites: Modules 33-35, 46-47.

ID	Lesson	Learning outcome
48.1	Formal properties, abstraction and assume-guarantee reasoning	Prove useful properties without hiding failures in assumptions.

ID	Lesson	Learning outcome
48.2	Clock/reset crossings and asynchronous FIFO verification	Check digital protocol correctness while acknowledging analog risk.
48.3	Fault models, ECC, testability and recovery	Design for detectable failures rather than assuming perfect hardware.
48.4	Hardware security, thermal limits and assurance review	Include isolation and failure containment in the design contract.

Module project: Create a verification dossier for the hardware capstone: requirements-to-tests traceability, assertions, coverage, clock/reset review, fault injection and a clearly bounded assurance claim.

References: R50, R49, R39, R51 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 49: Master's thesis: proposal, implementation and evaluation

Prerequisites: Gate I and relevant advanced-track modules; independent technical mentorship strongly recommended.

ID	Lesson	Learning outcome
49.1	Question selection, literature map and thesis scope	Turn a broad interest into a falsifiable, feasible research question.
49.2	Proposal review, baselines and risk-reduction milestones	Make failure cheap before the expensive implementation phase.
49.3	Implementation, ablations and controlled evaluation	Execute the study without changing the question to fit a favorable result.
49.4	Thesis writing, reproducibility and contribution boundaries	Write a technical argument another engineer can audit.

Module project: Prepare and execute a thesis-scale project in one primary specialization. Budget 400-800 focused hours as a planning range, with scope adjusted through review. The deliverable is a defensible contribution and reproducible artifact, not a promise of publication.

References: R42, R44, R48, R49, R55 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Module 50: Thesis defense, professional practice and continued development

Prerequisites: Module 49 and all prerequisites for the selected specialization.

ID	Lesson	Learning outcome
50.1	Oral defense and independent artifact reproduction	Show that the learner owns the reasoning and implementation.
50.2	Mature-codebase integration, review and maintainable contributions	Transfer a prototype into an engineering environment with other owners.
50.3	Portfolio, technical interviews and role-specific evidence	Present concrete engineering competence without inflated claims.
50.4	Long-term learning, maintenance and final handoff	Keep the knowledge and artifacts useful after the course ends.

Module project: Gate J: defend the thesis, pass an independent artifact reproduction, complete a maintainable handoff and publish an honest portfolio where rights permit. This completes the self-study program, not an accredited degree.

References: R01, R02, R05, R11, R49 | Four labs and 12 quiz questions, plus the project and applicable additional exercises.

Requirements outside the numbered lessons

Prerequisites and sequencing

The default sequence is numerical: 01.1 through 50.4. The digits before the decimal identify the module; the final digit identifies its lesson. Each module's opening page states prerequisites. Complete the practical checkpoint at the end of each volume before using its results as a foundation for the next stage.

Mathematics can run in parallel with early programming after Module 02, but the programming prerequisites of individual math labs must still be met. Do not begin advanced graphics without vectors, transforms and the relevant calculus/probability. Do not begin concurrency without memory and lifetime reasoning. Do not begin physical board work before the circuits and safety review. Graduate modules require the undergraduate evidence, not only recognition of terminology.

When a prerequisite is weak, return to its lesson and solve a new problem. Use a prerequisite repair log: missing skill, smallest review source, transfer exercise, result and date. A tutor or peer reviewer can help identify the actual gap. Starting at the beginning is not required for someone who can demonstrate the listed work independently.

Entry diagnostic: locate the starting point

Complete these without searching or using AI. This is placement information, not a pass/fail judgment. The answers are in the answer-key appendix under “Entry diagnostic,” separate from the 600 lesson questions.

1. Evaluate 3 + 4 x 2 using ordinary arithmetic precedence.

2. Convert 0.25 seconds to milliseconds.

3. Solve 3x + 2 = 14.

4. Evaluate 2^8.

5. Add vectors (-2,5) and (3,-1).

6. Find the length of vector (3,4).

7. Start at position 4, move right 3, then left 1. What is the final position?

8. For integer values: x=2; y=x+3; x=9. What is y?

9. What values does Python range(1,4) supply?

10. In the list [10,20,30], what is the value at zero-based index 1?

11. A while loop repeats while remaining>0 but never changes remaining or exits. What can go wrong?

12. Evaluate True and False.

13. A door opens at level 5 or above. Give three particularly useful boundary-test levels.

14. A program cannot find a relative filename. What environment fact should be checked first?

15. A program is fast on one input but produces a wrong result on another supported input. Is that an acceptable optimization?

With fewer than five correct answers on questions 1-6, schedule an algebra/trigonometry bridge using OpenStax Precalculus 2e while beginning Volume 1. Start with arithmetic, equations, functions, exponents, coordinate geometry and trigonometry; then retest. Questions 5-6 can also be learned in Module 07. This

result must not block the first programming session. [R40]

With fewer than five correct answers on questions 7-12, begin at Lesson 01.1. Strong results can justify moving faster, but not skipping the labs automatically. Questions 13-15 probe engineering habits that should develop throughout the course.

Prior-learning challenge process

A self-reported skill list is not enough to skip a volume. Attempt its lesson quizzes without notes, complete the checkpoint artifact or a comparable existing artifact, and explain a new variation to a reviewer. Require the same practical standards as the ordinary route. Record what was assessed and what was not.

For the math volume, the challenge must include a proof, a transform derivation, calculus/ODE reasoning and a probability or statistical interpretation; a multiple-choice vocabulary test is insufficient. For systems, include a lifetime bug, a concurrency argument, a network failure and a measured performance explanation. For digital design, include RTL semantics, interface verification, setup/hold reasoning and an explicit reset policy. Graduate credit within this self-study plan requires research-level evidence, not only professional seniority.

The study loop and assessment rules

For each lesson: retrieve prerequisite ideas from memory, read the teaching note, predict the worked example, complete the assigned source study, perform the lab, answer the quiz, then check the key. Save evidence rather than only checking a completion box. Required source readings are identified by R-codes linked in the resource directory. A chapter's explanation does not replace its exercises.

A suggested study block is 60-90 minutes with a break: roughly 10 minutes recall, 20-30 minutes reading/derivation, 25-40 minutes implementation or problems, then a brief journal entry. This is a planning suggestion, not a medical or cognitive prescription. Use a pace and breaks appropriate to the learner.

Each short quiz has three questions scored 0-2 each. Use 0 for absent/incorrect, 1 for partial or unexplained and 2 for correct with reasoning. A formative pass is **5/6 or better**, completed assigned study and the practical pass condition. The questions are deliberately short; they are not psychometrically validated exams or sufficient evidence of degree-level mastery by themselves.

For each missed concept, write the misconception and solve a changed example. Recheck after 24-72 hours when practical. At the end of a module, demonstrate a new input, requirement or failure without following the original walkthrough. An evaluator may accept a different technically sound answer than the key when the assumptions and reasoning are explicit.

Required depth beyond the short quizzes

At minimum, each programming/systems module should produce the stated project, **6-10 additional exercises**, and **five new edge or failure cases** beyond copied examples. Every mathematics module should include **12-20 written problems**, at least two derivations or proofs where appropriate, and a computational check. Select problems covering each named reading topic; use official solutions only after an independent attempt.

For graphics and simulation modules, preserve reference images or trajectories, numerical tests, a performance/quality comparison and one failure case. For digital-design modules, include a written interface, directed boundary tests, reproducible randomized tests, assertions, and tool reports appropriate to that module. For each graduate technical module, add **6-10 substantial analytical problems**, a critical reading of at least one original paper or equivalent graduate chapter, and a reproducible experiment with a baseline.

These are minimum planning requirements, not a reason to stop when understanding is incomplete. A mentor may substitute a comparably demanding assignment. Problems should come from the referenced course or book where possible and be logged by source, section and exercise number; this avoids pretending a one-page introduction replaces a semester's mathematical practice.

The mathematics extensions named in Volume 2 mean: R40 for algebra/trigonometry, R41 for multivariable derivatives and constrained differentials, and R42 for the later graduate optimization treatment. Complete the relevant R41 sections before gradient-based graduate work. Use R54 for deeper numerical methods. [R40-R42, R54]

For hardware/firmware direction, add a mandatory C-language bridge after Module 15: compile a small C source file as C, expose a C-compatible interface to C++, document allocation/free ownership, inspect structure layout and test error returns. C and C++ are distinct languages; do not treat compiling C code as C++ as proof of compatibility. Review the chosen compiler's C documentation and the target RTOS examples before Module 36. [R11, R53]

Review rubrics and academic integrity

Use this common project rubric, adapted to the project: correctness and requirements 30 points; tests/verification 25; reasoning and design tradeoffs 15; quantitative quality/performance/error analysis 10; reproducibility 10; communication and provenance 10. Suggested pass: **80/100**, with all essential practical checks complete and no unresolved critical defect inside the claimed operating envelope. A high visual-quality score cannot compensate for an unsafe memory interface or a false correctness claim.

For graduate artifacts: contribution/question clarity 10; mathematical or technical rigor 20; implementation correctness 20; evaluation/baselines/uncertainty 25; reproducibility 15; writing and defense 10. Suggested pass: 80/100 plus independent reproduction of the central claim. Publication acceptance and positive results are not required. Clear scope, honest failure analysis and defensible evidence are required.

Use AI as a tutor, reviewer or brainstorming aid, not an unexamined answer source. Predict before asking for code, identify generated sections, check provenance, test outputs and explain the implementation. Never upload private employer code, proprietary SDK material, credentials or other people's personal data without authorization. Supplied starter code is a learning reference; running it does not complete the learner's own project requirements.

Thesis project menu: choose one primary question

Area	Example question	Essential evidence
Rendering	Can a sampling/reconstruction change reduce equal-error time on a defined moving-scene set?	Estimator derivation, reference images, motion failures, total cost and uncertainty.
Engine scheduling	Can a bounded scheduling policy reduce p99 frame time during streaming bursts without exceeding memory limits?	Task traces, competent baselines, burst workloads, memory peaks and ablations.
Physics	How do timestep, iteration count and compliance affect error and cost in a small deformable simulation?	Equations, gradient/constraint checks, reference solution and sensitivity study.
GPU algorithms	Does a compaction or wavefront strategy improve a specified irregular workload after queue overhead?	Work/span reasoning, termination and bounds tests, skewed workloads and end-to-end timings.
Accelerator design	Does a verified fixed-point block meet a game-quality tolerance and improve end-to-end latency or energy?	Numerical error, RTL tests/properties, host interface, PPA assumptions and transfer cost.
Architecture / compilers	Does a cache, layout or specialization policy improve a stated game-workload family?	Model validation, semantic correctness, held-out workloads and Pareto tradeoffs.

These are question templates, not claims that the techniques are new or guaranteed to succeed. Review relevant newer research when the project begins. The 2016 XPBD and 2020 ReSTIR references are stable historical anchors for reproduction, not a survey of the latest work. [R44, R55]

Hardware, software and safety policy

Start with the computer already available. The first-day Python work and early programming do not require a dedicated GPU, FPGA, oscilloscope, cloud subscription or game-engine license. Use a code editor, a supported Python 3 interpreter and later Git. At Module 11, add a C++20-capable compiler and CMake. Record the actual versions rather than upgrading tools in the middle of a reproducibility experiment. [R04, R05, R11, R12]

At graphics, choose one supported path: Vulkan, native Direct3D 12 on a compatible Windows setup, or Metal on a compatible Apple setup. Check actual device and tutorial capabilities before installation. A moving “latest” tutorial may assume newer features than the learner's device supports. SDL3 examples must use SDL3 APIs rather than mixing SDL2 instructions. Keep the CPU renderer as a reference and fallback. [R21, R22, R24, R25]

CUDA is an optional vendor-specific route, not a requirement for learning GPU algorithms. Use compute shaders or a supported alternative where appropriate, and explicitly limit claims when a GPU is unavailable. Use official platform documentation rather than leaked console SDKs. No supplied lesson grants proprietary access or distribution rights. [R21, R24, R25, R37]

For digital design, begin with supported RTL simulation and synthesis tools such as Verilator, cocotb, Yosys and SymbiYosys; compatibility depends on versions and language features. Board selection should follow the project and toolchain, not precede them. For architecture and physical design, use gem5 and an educational OpenROAD-compatible flow only within their documented model and library scope. [R32-R34, R38, R39, R50]

Physical experiments require a qualified mentor, exact board/peripheral documentation, compatible voltage levels and a low-voltage safety plan. Keep protective mechanisms enabled. Do not open or modify mains

power supplies, CRTs, high-energy battery packs or high-voltage equipment. Do not design a first project around mains circuitry. Do not assume a low nominal voltage removes short-circuit or thermal risks. Use reviewed commercial educational boards and approved supplies. Hardware procurement and fabrication are optional extensions, not first-day requirements.

Source use, maintenance and limitations

R-codes identify the authoritative reading directory that follows. The lesson teaching notes and quizzes are original instructional synthesis; cited resources provide primary course material, official specifications, author textbooks or original research. Resource descriptions are not claims that every linked page is a complete beginner course. For example, the Python tutorial is a reference for people with some programming familiarity; CS50 Python is the recommended beginner lecture companion. [R03, R04]

Use the source sections named in each lesson, and log the actual chapter and problem numbers studied. Some course sites are historical archives, intentionally chosen for durable foundations. Software documentation is often a moving target. Record a release or access date and use matching documentation. Where an exact page moves, navigate from the organization's official documentation rather than silently replacing it with an unreviewed tutorial.

The source directory was assembled for this edition dated September 28, 2026. It is not a guarantee that every future link, tool feature or licensing term remains unchanged. Original books, papers, software and courses retain their own licenses. They are linked, not included as copied books or bundled proprietary software. All hardware and performance examples must be revalidated for the learner's chosen system.

This curriculum emphasizes essential technical depth for game engines and gaming hardware. It does not replace a university's full general-education requirements, supervised laboratories, formal academic assessment or the specialized mentorship required for production ASIC signoff. Its strength is a coherent sequence of explicit learning outcomes and evidence-producing work, not a credential claim.

Volume checkpoints: Gates A-J

These exact checkpoint requirements come from the original Program Guide. Each follows the corresponding volume and overlaps its culminating module project.

Gate A: a complete first game

After Volume 1. Build a terminal grid or resource game with a clear initial state, commands, valid/invalid input behavior, a success/failure condition and restart. Keep the simulation separate from display. Use the included game as a reference, then implement meaningful changes rather than submitting it unchanged.

Deliver the source, README, a rule specification, at least 12 learner-written tests, a saved input sequence and its expected final state, and a short journal explaining two bugs. Include tests for boundaries, blocked movement, invalid commands, restart and repeated rendering without state changes. Replaying the same inputs in the pinned environment must produce the expected result. A second person should run a complete loop from the README.

The oral check asks the learner to change one rule, predict affected tests and repair the implementation. Passing does not require graphics. Do not add networking, a database or a large engine merely to make the project look advanced.

Gate B: a game-mathematics notebook

After Volume 2. Produce a versioned notebook or scripts plus written derivations. Required artifacts are a logic/invariant proof, vector and matrix operations, a transform-chain example, a one-dimensional dynamics model, a sampling/aliasing demonstration and a small statistically analyzed experiment.

Tests must include a known transform, a zero-vector edge case, an analytic motion comparison, a timestep refinement study and a probability calculation checked by simulation. Show units and numerical tolerances. Do not normalize a zero vector silently or treat a random experiment as proof. Complete and log the required source problem sets.

The oral check supplies a changed coordinate convention, a new timestep and a misleading average. The learner must explain what changes and what remains invariant. A screenshot of a plot without derivation or reproducible data is insufficient.

Gate C: native simulation and a tiny language

After Volume 3. Implement the core simulation in C++ with explicit ownership and a build system, then add a small expression language or bytecode interpreter. It may evaluate movement or material expressions; it must have a specified grammar, type/error behavior and resource limits.

Deliver a clean build, tests, memory/lifetime checks available on the toolchain, a complexity analysis and a comparison of two appropriate data structures. Include empty input, malformed tokens, invalid bytecode or expressions, lifetime boundaries, container invalidation and numerical limits. Add the C-language bridge for the hardware direction.

The oral check includes one pointer/lifetime defect, a data-structure choice and a new grammar case. The learner should explain the entire compile/link/run path. A library may supply support utilities, but not replace the central learning implementation without an explicit comparison exercise.

Gate D: a defensible systems application

After Volume 4. Build a local job-based application that reads validated work, performs bounded processing and stores results. A small local client/server or task-processing tool is enough; public deployment is unnecessary.

Deliver a requirements sheet, architectural decision record, thread/resource ownership map, protocol/schema, threat model, automated build/test script, dependency inventory and recovery report. Test malformed messages, duplicate requests, cancellation, partial writes, worker failure and shutdown under load. Run relevant memory/concurrency diagnostics where supported and record their limitations.

The oral check asks why the design is race-free under its stated model, how retries avoid duplicated durable effects and what evidence identifies the measured bottleneck. A functioning happy path alone does not pass.

Gate E: a small 3D engine

After Volume 5. Build an engine demonstration containing a CPU reference renderer, one supported GPU backend where available, a fixed-step simulation, explicit entity/resource ownership, an asset cooker, an inspector and versioned save/load. A software-only execution path is acceptable while GPU access is unavailable, but untested GPU claims must remain excluded.

Deliver a fixed scene, source assets with provenance, build/cook commands, validation logs, headless tests, screenshot or image-difference evidence, frame-time data and a peak-memory report. The budget is chosen for the declared test machine; 60 fps is an example target, not a universal pass requirement. Test near-plane crossing, resize/minimize, stale handles, malformed assets and resource shutdown.

A second person must build from a clean checkout or extracted source tree. The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. Using an existing engine for comparison is encouraged; using it in place of the central engine implementation changes the learning objective and needs a separate implementation project.

Gate F: undergraduate software capstone

After Volume 6. Release a small original game built on the engine. Include one finished gameplay loop; collision/physics; animation or articulated motion; bounded audio; an inspectable AI behavior; settings; save/load; and a tested package. The multiplayer requirement can be a separate local two-client demonstration with delay, loss and replay tests rather than a public online service.

Evidence includes a compatibility matrix, remappable controls, readable interface tests, alternative presentation for a critical audio cue, pseudo-localization, a clean-folder launch and a technical postmortem. Preserve CPU/GPU or software-renderer timing boundaries, raw frame distributions and memory measurements. Test interrupted saves, invalid mods if modding is exposed, controller/device absence and exit during loading.

A reviewer must finish the loop, reproduce a bug test and observe a live change. The quality bar is a coherent, testable small system. It is not commercial art quality, a large open world or a revenue target.

Gate G: undergraduate digital-design capstone

After Volume 7. Implement one RTL subsystem: a synchronous queue with a small processor interface, a sprite compositor, a fixed-point vector block or another similarly bounded game-related function. The module-level processor exercise remains required even when the capstone centers on a different block.

Deliver the interface specification, software reference, RTL, automated testbench, directed and randomized cases, assertions, reset review and synthesis reports. Test empty/full, simultaneous operations, backpressure, overflow/rounding and reset while work is pending where applicable. Label simulation, synthesis and physical measurements separately.

No board purchase is required. A board demonstration is optional and follows the safety plan. Passing FPGA operation does not establish ASIC readiness. The oral check asks for a waveform explanation, a nonblocking-assignment trace and setup versus hold reasoning.

Gate H: undergraduate hardware/software co-design capstone

After Volume 8. Choose one measured engine workload. Compare a competent software baseline with at least two architecture alternatives, implementing one alternative as RTL, a supported simulation model or a bounded analytical prototype with clear limitations.

Required evidence includes workload distributions, numerical-quality limits, host/device ownership, setup and transfer costs, resource/timing estimates, failure handling and a sensitivity analysis. Evaluate small and large workloads and one adverse case. A negative conclusion is acceptable: proving that a software optimization is preferable is valid co-design work.

The review must identify the maximum application benefit, the dominant bottleneck, which results are modeled and which are measured, and what changes under a tighter memory or latency constraint. Do not compare an optimized accelerator against an intentionally weak software baseline.

Gate I: graduate reproduction and extension

After Volume 9. Select a bounded published or graduate-level method in rendering, simulation, GPU algorithms or compilation. Reconstruct the method, reproduce one central result and add a clearly identified extension, new setting or rigorous failure analysis. The extension need not be globally novel; its relation to prior work must be honest.

Deliver a claim-evidence map, derivation or semantic argument, implementation, independent reference or baseline, raw data, analysis scripts, one ablation and one parameter sensitivity study. Use equal-quality comparisons when quality changes with speed. Report uncertainty at the appropriate trial unit and avoid training/tuning on the final evaluation set without disclosure.

A reviewer should reproduce the main table or result. Unresolved discrepancies must be explained and narrow the claim. This checkpoint is the prerequisite for the thesis, not a replacement for it.

Gate J: thesis, defense and handoff

After Volume 10. Complete one primary thesis-scale project. Plan roughly 400-800 focused hours within the volume's estimate, but adjust the scope through review rather than treating the hours as a requirement to spend. A 30-60 page thesis-style report plus technical appendices is a useful planning target; evidence quality matters more than length.

Deliver a literature map, reviewed proposal, running baseline, risk register, versioned decisions, final method, tests, evaluation, reproducible artifact and maintainer handoff. Hold a 30-45 minute defense with a qualified reviewer or peer panel, including a derivation, a failed case and a live implementation question. Independently reproduce the central result and complete revisions.

One thesis is enough. The learner need not invent both a new engine and a new GPU. Completion records should say self-study curriculum and describe the actual artifacts, not claim an accredited bachelor's or master's degree.

Reusable templates

File	Purpose
templates/ARCHITECTURE_DECISION.md	Record context, alternatives, decision, consequences and evidence.
templates/BENCHMARK_MANIFEST.md	Record workload, environment, measurement method, raw data and reproduction details.
templates/HARDWARE_LAB_RECORD.md	Record the hardware setup, safety review, predictions, verification and evidence tier.
templates/LAB_JOURNAL.md	Record predictions, work, results, failures and next steps.
templates/PAPER_REVIEW.md	Map research claims, assumptions, method, evidence and reproduction questions.
templates/PREREQUISITE_REPAIR.md	Track a missing skill, review source, transfer exercise and reassessment.
templates/PROJECT_REVIEW.md	Review project evidence, correctness, verification and remaining issues.
templates/THESIS_OUTLINE.md	Structure the thesis question, prior work, method, evaluation and handoff.

Starter package

The archive supplies Python examples and tests, a small C++/CMake example, an environment check, a tested-environment record and usage instructions. These are starter references, not solutions to the 200 labs or completed project submissions.

Source and progress files

The source folder contains ten editable volume files, guide/start/answer/reference Markdown, curriculum.json and browser.js. CURRICULUM.html provides search, expandable answers and local progress export/import. The full 299-page curriculum excludes the separate 52-page answer key; page counts here follow the original archive manifest.

Full reference directory

The following 55 entries are reproduced from the package. This manifest does not reverify the links or update tool versions. External publications and software retain their own rights; their full contents are not bundled.

R01: CS2023 knowledge areas

ACM / IEEE-CS / AAAI

Breadth checklist, not a claim of accreditation.

https://csed.acm.org/knowledge-areas/

R02: Computer Engineering Curricula 2016

ACM / IEEE Computer Society

Computer-engineering breadth and laboratory expectations; historical curriculum report.

https://ieeecs-media.computer.org/assets/pdf/ce2016-final-report.pdf

R03: CS50 Introduction to Programming with Python

Harvard University

Beginner lectures and practice; complete exercises rather than only watching.

https://cs50.harvard.edu/python/

R04: Python tutorial and language documentation

Python Software Foundation

Reference companion; the tutorial itself assumes some programming familiarity.

https://docs.python.org/3/tutorial/

R05: Pro Git

Scott Chacon and Ben Straub

Version control and collaboration.

https://git-scm.com/book/en/v2

R06: 6.042J Mathematics for Computer Science

MIT OpenCourseWare

Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R07: 18.06 Linear Algebra

MIT OpenCourseWare

Matrices, vector spaces, eigenvalues and problem sets.

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

R08: 18.01SC Single Variable Calculus

MIT OpenCourseWare

Calculus lectures, exercises and assessments.

https://ocw.mit.edu/courses/18-01sc-single-variable-calculus-fall-2010/

R09: 18.05 Introduction to Probability and Statistics

MIT OpenCourseWare

Probability, inference and quantitative reasoning.

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

R10: 6.003 Signals and Systems

MIT OpenCourseWare

Sampling, convolution, transforms and systems.

https://ocw.mit.edu/courses/6-003-signals-and-systems-fall-2011/

R11: C++ Core Guidelines

C++ Core Guidelines editors

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12: CMake tutorial

Kitware

Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13: 6.006 Introduction to Algorithms

MIT OpenCourseWare

Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14: Kaleidoscope and LLVM tutorials

LLVM Project

Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15: 6.004 Computation Structures

MIT OpenCourseWare

Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16: Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau

Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R17: RFC 9293: Transmission Control Protocol

IETF / RFC Editor

Transport semantics; reference reading after networking foundations.

https://www.rfc-editor.org/rfc/rfc9293.html

R18: PostgreSQL tutorial

PostgreSQL Global Development Group

Relational data, transactions and SQL; pin local server and client versions.

https://www.postgresql.org/docs/current/tutorial.html

R19: 15-462/662 Computer Graphics

Carnegie Mellon University

Rendering, geometry and animation course structure and exercises.

https://15462.courses.cs.cmu.edu/fall2021/

R20: 15-466/666 Computer Game Programming

Carnegie Mellon University

Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

R21: Vulkan tutorial

Khronos Group

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

R22: SDL3 documentation

SDL project

Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples.

https://wiki.libsdl.org/SDL3/FrontPage

R23: Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys

Rendering mathematics and implementation; use chapter titles in lesson assignments.

https://www.pbr-book.org/4ed/contents

R24: Direct3D 12 graphics

Microsoft

Windows graphics backend reference.

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

R25: Metal developer resources

Apple

Apple-platform graphics and compute reference.

https://developer.apple.com/metal/

R26: Programming with C++ in Unreal Engine

Epic Games

Engine comparison after C++ foundations, not a beginner substitute.

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

R27: Fix Your Timestep!

Glenn Fiedler

Original practitioner explanation of fixed-step simulation; read critically and implement your own lab.

https://gafferongames.com/post/fix_your_timestep/

R28: Box2D documentation

Box2D project

Collision, constraints and simulation reference.

https://box2d.org/documentation/

R29: Xbox Accessibility Guidelines

Microsoft

Accessibility design and testing; not a legal-compliance certificate.

https://learn.microsoft.com/en-us/xbox/accessibility/guidelines

R30: 6.002 Circuits and Electronics

MIT OpenCourseWare

Circuits, transistors and laboratory reasoning.

https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/

R31: Nand to Tetris

Noam Nisan and Shimon Schocken

Computer construction projects; review the authors' license terms.

https://www.nand2tetris.org/

R32: Verilator user guide

Verilator project

RTL simulation and linting; match supported SystemVerilog subset.

https://verilator.org/guide/latest/

R33: Yosys documentation

YosysHQ

Synthesis and technology mapping; stable installed release preferred over a development snapshot.

https://yosyshq.readthedocs.io/projects/yosys/en/latest/

R34: cocotb documentation

cocotb project

Python-based HDL verification; simulator and version compatibility must be checked.

https://docs.cocotb.org/en/stable/

R35: RISC-V ISA specifications

RISC-V International

Instruction-set reference; record the exact ISA version and implemented extensions.

https://docs.riscv.org/reference/isa/

R36: KiCad documentation

KiCad project

Schematic capture, PCB layout and manufacturing outputs.

https://docs.kicad.org/

R37: CUDA Programming Guide

NVIDIA

GPU execution and memory; CUDA is an optional vendor-specific lab route.

https://docs.nvidia.com/cuda/cuda-programming-guide/

R38: gem5 documentation

gem5 project

Architecture simulation; distinguish model outputs from measured hardware results.

https://www.gem5.org/documentation/

R39: OpenROAD documentation

OpenROAD project

RTL-to-physical-design study; educational results are not production signoff.

https://openroad.readthedocs.io/en/latest/

R40: Precalculus, second edition

OpenStax / Rice University

Algebra, functions and trigonometry bridge before calculus.

https://openstax.org/details/books/precalculus-2e

R41: 18.02SC Multivariable Calculus

MIT OpenCourseWare

Partial derivatives, gradients, multiple integrals and constrained differentials.

https://ocw.mit.edu/courses/18-02sc-multivariable-calculus-fall-2010/

R42: Convex Optimization

Stephen Boyd and Lieven Vandenberghe

Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

https://web.stanford.edu/~boyd/cvxbook/

R43: Roofline performance model, technical report

Samuel Williams, Andrew Waterman and David Patterson

Original 2008 performance-model reference; use current measured hardware limits for experiments.

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

R44: Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Benedikt Bitterli and coauthors

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

R45: AddressSanitizer documentation

LLVM Project

Memory-error instrumentation, limitations and supported toolchains.

https://clang.llvm.org/docs/AddressSanitizer.html

R46: CS188 Introduction to Artificial Intelligence textbook

University of California, Berkeley

Search, games, decision processes, learning and probabilistic reasoning.

https://inst.eecs.berkeley.edu/~cs188/textbook/

R47: Dive into Deep Learning

Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola

Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions.

https://d2l.ai/

R48: 6.823 Computer System Architecture

MIT OpenCourseWare

Graduate architecture foundations; historical course concepts supplemented with current specifications.

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

R49: ECE5745 Complex Digital ASIC Design

Christopher Batten / Cornell University

Graduate ASIC methodology, circuits, EDA and design-project expectations.

https://www.csl.cornell.edu/courses/ece5745/

R50: SymbiYosys documentation

YosysHQ

Formal hardware verification, safety, liveness and cover properties.

https://yosyshq.readthedocs.io/projects/sby/en/latest/

R51: Driver implementer API guide

Linux kernel contributors

Device ownership, I/O, DMA and driver interfaces; use disposable VMs or simulated devices for kernel experiments.

https://docs.kernel.org/driver-api/index.html

R52: OpenXR registry and specifications

Khronos Group

XR coordinate spaces, frame timing and runtime interfaces; optional hardware-dependent extension.

https://registry.khronos.org/OpenXR/

R53: Zephyr documentation

Zephyr Project

Embedded kernel, drivers, board support and simulation; select and pin a supported release.

https://docs.zephyrproject.org/latest/introduction/index.html

R54: 18.085 Computational Science and Engineering I

MIT OpenCourseWare

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

R55: Author publication archive: position-based simulation and XPBD

Matthias Mueller and collaborators

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

https://matthias-research.github.io/pages/publications/publications.html

Complete original archive inventory

Exactly 54 files, relative to game_engineering_curriculum/. The newly created manifest files are not part of that count. The original ZIP was not modified. Exact SHA-256 checksums are in the companion JSON.

Package root

ANSWER_KEY.pdf (146,922 bytes | 52 pages)
COMPLETE_CURRICULUM.pdf (964,658 bytes | 299 pages)
CURRICULUM.html (648,679 bytes)
MANIFEST.json (3,182 bytes)
PROGRAM_GUIDE.pdf (120,687 bytes | 21 pages)
README.md (3,587 bytes)
START_HERE.pdf (82,474 bytes | 8 pages)
VALIDATION.md (2,261 bytes)

source/

source/01_foundations_and_your_first_game.md (32,633 bytes)
source/02_mathematics_for_games_and_computing.md (32,440 bytes)
source/03_c_algorithms_and_language_implementation.md (32,771 bytes)
source/04_computer_systems_and_software_engineering.md (32,736 bytes)
source/05_graphics_and_game_engine_construction.md (35,991 bytes)
source/06_game_systems_and_production.md (36,102 bytes)
source/07_computer_engineering_and_digital_design.md (35,277 bytes)
source/08_gaming_platforms_and_hardware_software_co_design.md (36,164 bytes)
source/09_graduate_game_engine_and_graphics_engineering.md (37,931 bytes)
source/10_graduate_gaming_hardware_and_research_thesis.md (38,222 bytes)
source/ANSWER_KEY.md (64,018 bytes)
source/PROGRAM_GUIDE.md (30,937 bytes)
source/REFERENCES.md (10,612 bytes)
source/START_HERE.md (10,109 bytes)
source/browser.js (4,172 bytes)
source/curriculum.json (485,658 bytes)

starter/

starter/README.md (2,661 bytes)
starter/TESTED_ENVIRONMENT.md (777 bytes)
starter/day01_motion.py (313 bytes)
starter/environment_check.py (427 bytes)
starter/grid_game.py (3,393 bytes)
starter/simulation.py (1,240 bytes)
starter/test_grid_game.py (1,915 bytes)
starter/test_simulation.py (1,717 bytes)

starter/cpp/

starter/cpp/CMakeLists.txt (526 bytes)
starter/cpp/advance.hpp (645 bytes)
starter/cpp/main.cpp (353 bytes)
starter/cpp/tests.cpp (1,758 bytes)

templates/

templates/ARCHITECTURE_DECISION.md (613 bytes)
templates/BENCHMARK_MANIFEST.md (771 bytes)
templates/HARDWARE_LAB_RECORD.md (1,031 bytes)
templates/LAB_JOURNAL.md (838 bytes)
templates/PAPER_REVIEW.md (662 bytes)

templates/PREREQUISITE_REPAIR.md (492 bytes)
templates/PROJECT_REVIEW.md (938 bytes)
templates/THESIS_OUTLINE.md (901 bytes)

volumes/

volumes/01_foundations_and_your_first_game.pdf (115,545 bytes | 27 pages)
volumes/02_mathematics_for_games_and_computing.pdf (114,849 bytes | 27 pages)
volumes/03_c_algorithms_and_language_implementation.pdf (114,770 bytes | 27 pages)
volumes/04_computer_systems_and_software_engineering.pdf (117,447 bytes | 27 pages)
volumes/05_graphics_and_game_engine_construction.pdf (122,646 bytes | 27 pages)
volumes/06_game_systems_and_production.pdf (121,032 bytes | 27 pages)
volumes/07_computer_engineering_and_digital_design.pdf (121,507 bytes | 27 pages)
volumes/08_gaming_platforms_and_hardware_software_co_design.pdf (127,234 bytes | 27 pages)
volumes/09_graduate_game_engine_and_graphics_engineering.pdf (126,225 bytes | 27 pages)
volumes/10_graduate_gaming_hardware_and_research_thesis.pdf (126,181 bytes | 27 pages)

Validation

The inventory was checked directly against the original ZIP: 54 files, 50 modules, 200 unique lesson IDs and 600 lesson quiz questions. Each volume has five modules and each module has four lessons. PDF page counts were copied from the original manifest, not inferred from filenames. No course files were changed.